

Document technique final

Plateforme de gestion d'un cabinet d'audioprothèse
Solution DevOps hybride sur Microsoft Azure

Équipe	Zaafir Mougammadou Zaccaria · Elyess Rjafellah · Adame Nianghane · Anis Douadi
Période	Janvier – Juin 2026
Client	Cabinet d'audioprothèse « AudioPro » (commanditaire)
Dépôt	github.com/Rifaazs27/projet-audioprothese
Nature	Rendu de groupe — solution technique complète

SUP DE VINCI — Expert en Systèmes d'Information — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude du Mastère DevOps. Code promo : **CODEPROMO** (à compléter par l'équipe).

1. Présentation de l'entreprise et de l'équipe

« AudioPro » est un cabinet d'audioprothèse qui accompagne ses patients tout au long de leur appareillage. Son activité quotidienne repose sur le suivi d'un ensemble d'informations sensibles : le dossier de chaque patient, les appareils auditifs qui lui sont posés, et le calendrier des rendez-vous de contrôle. Jusqu'ici gérées de façon dispersée, ces données méritaient une plateforme centralisée, fiable et sécurisée, capable de garantir la confidentialité exigée par la réglementation sur les données de santé tout en restant disponible et simple d'usage pour les praticiens.

Le cabinet a confié à notre équipe la conception et la mise en production de cette plateforme, non pas comme une simple application isolée, mais comme un **système complet exploité selon les pratiques DevOps** : automatisation des déploiements, infrastructure décrite sous forme de code, supervision permanente, sécurité intégrée à chaque étape et capacité à se rétablir après incident. L'objectif pédagogique sous-jacent était de démontrer notre maîtrise de l'ensemble de la chaîne, du code source jusqu'à l'exploitation en conditions réelles.

Notre équipe est composée de quatre étudiants du Mastère DevOps. Afin de couvrir l'intégralité du périmètre sans recouvrement inutile, nous avons réparti les responsabilités par grands domaines techniques, chacun étant pilote sur son sujet tout en contribuant aux revues de code et aux décisions d'architecture communes :

Membre	Rôle	Périmètre principal
Zaafir Mougammadou Zaccaria	Infrastructure & déploiement	Infrastructure as code Terraform (AKS, ACR, PostgreSQL, réseau, budget), application FastAPI/React, déploiement applicatif.
Elyess Rjafellah	CI/CD & automatisation	Chaîne GitHub Actions (provisionnement, build, scans, déploiement, sauvegarde), état Terraform distant, pipeline auto-réparant.
Adame Nianghane	Observabilité & sécurité	Supervision Prometheus/Grafana/Loki/Alertmanager, DevSecOps (Trivy, CodeQL, Gitleaks), RBAC, TLS, conformité RGPD.
Anis Douadi	Kubernetes & on-premise / PRA	Packaging Helm, autoscaling et politiques, site on-premise simulé, MinIO chiffré, Ansible, plan de reprise d'activité.

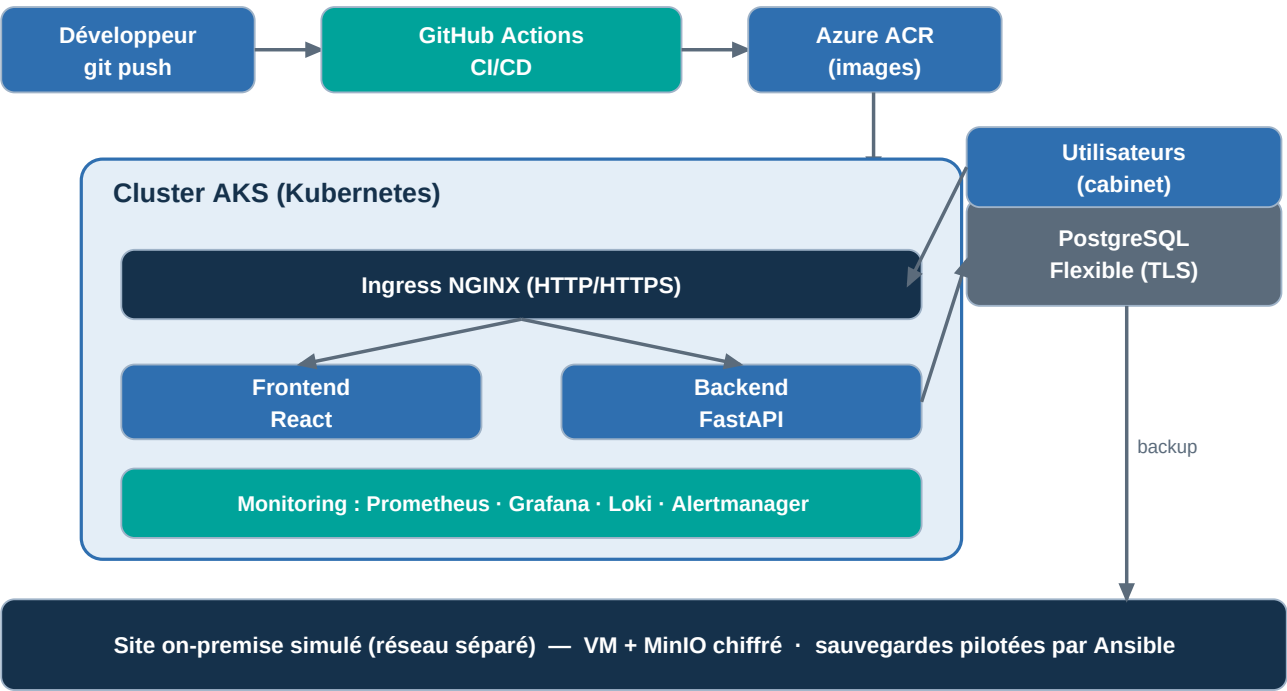
2. Analyse de la problématique et solution proposée

La problématique dépasse le simple développement d'un logiciel. Les données manipulées étant des données de santé, la **confidentialité** et la **traçabilité** sont impératives. Le service devant être utilisé au quotidien par le cabinet, sa **disponibilité** et sa capacité d'**auto-rétablissement** sont essentielles. La solution doit en outre rester **économique** — le projet s'inscrit dans le cadre d'un compte Azure for Students plafonné à 85 dollars — et entièrement **reproductible**, afin de pouvoir être reconstruite à l'identique à tout moment.

Pour répondre à ces exigences, nous avons conçu une application web conteneurisée — une interface React, une API FastAPI et une base PostgreSQL managée — déployée sur un cluster Kubernetes managé (Azure AKS) et encadrée par une chaîne DevOps de bout en bout.

L'infrastructure est décrite en Terraform, les déploiements sont automatisés par GitHub Actions, la supervision repose sur Prometheus, Grafana et Loki, et la sécurité est vérifiée automatiquement à chaque livraison. Conformément au cahier des charges, l'architecture est **hybride** : le cloud héberge l'application tandis qu'un site on-premise (simulé par un réseau séparé) accueille le stockage objet chiffré des sauvegardes, illustrant la gestion des données sensibles hors du cloud public.

Le schéma ci-dessous synthétise cette architecture, depuis le poste du développeur jusqu'à l'exécution supervisée en production et la réplication des sauvegardes vers l'on-premise :



Le flux est le suivant : chaque modification poussée par un développeur déclenche le pipeline d'intégration et de déploiement, qui teste le code, en vérifie la sécurité, construit les images, les publie dans le registre Azure Container Registry puis les déploie sur le cluster via Helm. L'application est exposée aux utilisateurs par un Ingress unique ; le backend dialogue avec la base PostgreSQL en TLS ; la supervision et les sauvegardes fonctionnent en continu et de façon autonome.

3. Gestion des coûts — démarche FinOps (M2)

La maîtrise des coûts a été traitée comme une contrainte de conception et non comme une préoccupation de fin de projet. Le budget disponible — 85 dollars de crédit étudiant, sans renouvellement — nous a imposé des arbitrages permanents entre exhaustivité technique et sobriété. Le tableau suivant présente l'estimation des coûts de l'infrastructure lorsqu'elle fonctionne en continu :

Ressource	Dimensionnement	Coût indicatif (24/7)
AKS — plan de contrôle	Free tier	0 \$
AKS — nœuds de calcul	2 × Standard_B2s_v2	≈ 60–70 \$ / mois
Base de données	PostgreSQL Flexible B1ms	≈ 13 \$ / mois

Ressource	Dimensionnement	Coût indicatif (24/7)
Registre d'images	Azure Container Registry Basic	≈ 5 \$ / mois
VM on-premise (MinIO)	Standard_B2s_v2	≈ 30 \$ / mois
Équilibres & IP publiques	Standard	≈ 6–9 \$ / mois

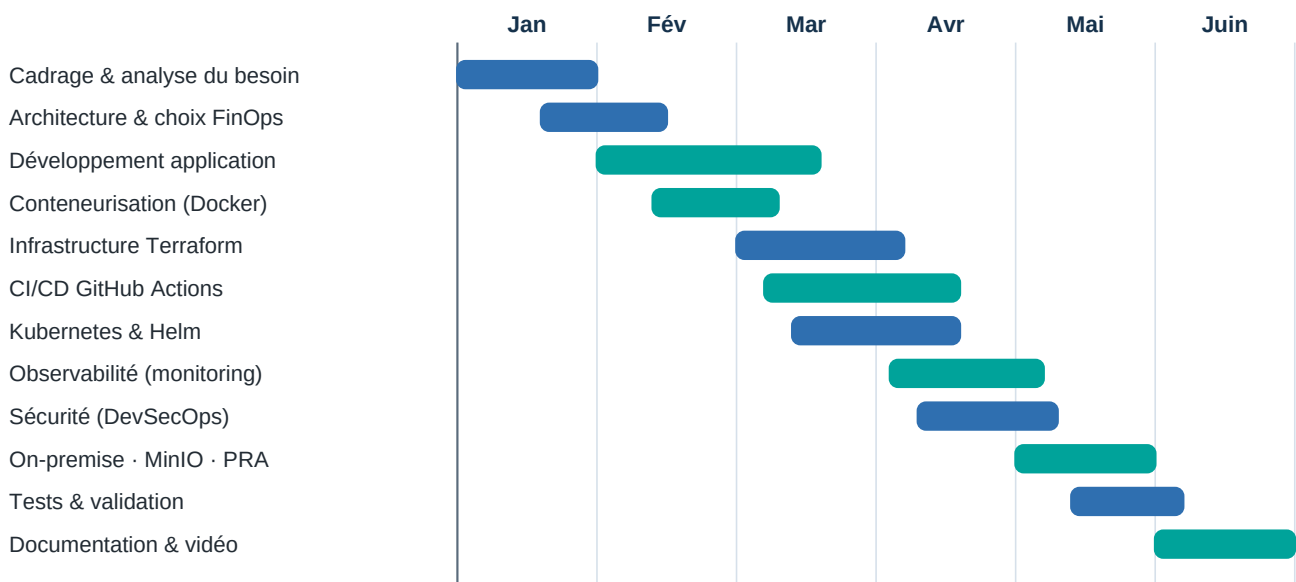
Plusieurs leviers d'optimisation ont été appliqués. Le plan de contrôle Kubernetes est gratuit et les nœuds reposent sur des machines « burstable » de la série B, peu coûteuses. Pour la centralisation des journaux, nous avons délibérément retenu Loki plutôt que la pile ELK : Loki n'indexe que les métadonnées et consomme une fraction des ressources d'Elasticsearch, ce qui était incompatible avec notre budget. Les secrets sont conservés dans GitHub Actions plutôt que dans un coffre hébergé en permanence, et les durées de rétention (métriques, sauvegardes) ont été réduites au strict nécessaire.

Surtout, l'infrastructure n'est pas destinée à tourner en permanence. Un budget Azure assorti d'alertes par courriel à 50, 80 et 100 % du plafond prévient toute dérive, et l'ensemble peut être **détruit en un clic** puis reconstruit en une douzaine de minutes. En pratique, l'environnement n'est allumé que pendant les phases de test et de démonstration, ramenant le coût réel à quelques dollars sur toute la durée du projet.

4. Organisation de l'équipe, planification et méthodologie

Nous avons adopté une organisation agile inspirée de Scrum et de Kanban, rythmée par des sprints de deux semaines et une priorisation des tâches selon la méthode MoSCoW. Le dépôt Git a servi de point central : chaque évolution passait par une branche puis une revue de code (Pull Request) avant d'être intégrée, garantissant la qualité et le partage de connaissances entre les membres. Les outils mobilisés sont représentatifs d'une chaîne DevOps professionnelle : Git et GitHub Actions pour l'intégration et le déploiement continu, Terraform et Ansible pour la gestion de configuration, et Helm pour le packaging Kubernetes.

Le projet s'est déroulé sur le second semestre de l'année universitaire, de janvier à juin 2026, selon une démarche agile en sprints de deux semaines. Le diagramme de Gantt ci-dessous retrace l'enchaînement des grandes phases, du cadrage initial à la rédaction des livrables et à l'enregistrement de la vidéo de démonstration.



La répartition des contributions, équilibrée entre les quatre membres, est détaillée ci-dessous. Chaque contributeur a piloté un domaine tout en participant aux décisions transverses ; la traçabilité individuelle est consultable dans l'historique du dépôt et la vue « Contributors » de GitHub.

Membre	Contributions concrètes au projet
Zaafir Mougammadou Zaccaria	Conception des modules Terraform (groupe de ressources, AKS, ACR, PostgreSQL, réseau, budget), développement de l'API FastAPI et du frontend, conteneurisation et déploiement applicatif Helm.
Elyess Rjafellah	Conception des workflows GitHub Actions (intégration, sécurité, déploiement, sauvegarde planifiée), automatisation du provisionnement, état distant et mécanismes d'auto-réparation du pipeline.
Adame Nianghane	Mise en place de la supervision (Prometheus, Grafana, Loki, Alertmanager) et des tableaux de bord, intégration des scans de sécurité dans la CI, durcissement (RBAC, TLS, secrets, conteneurs non-root).
Anis Douadi	Packaging Helm de l'application, autoscaling et politiques réseau, provisionnement de la VM on-premise, configuration de MinIO chiffré et des playbooks Ansible de sauvegarde et de restauration.

5. Présentation de la solution technique

La solution repose sur un ensemble cohérent de technologies, chacune choisie pour sa pertinence et son coût d'exploitation :

Domaine	Technologies retenues
Application	Python 3.11, FastAPI, SQLAlchemy, Pydantic ; React 18 + Vite
Données	PostgreSQL 16 (Azure Flexible Server, TLS obligatoire)
Conteneurs / Orchestration	Docker (images multi-stage non-root), Kubernetes (AKS), Helm
Infra & configuration	Terraform (provider azurearm), Ansible
CI/CD	GitHub Actions (déploiement, sauvegarde, intégration, sécurité)

Domaine	Technologies retenues
Observabilité	Prometheus, Grafana, Loki, Alertmanager
Sécurité	Trivy, CodeQL, Gitleaks, RBAC, NetworkPolicies, TLS, secrets chiffrés
On-premise / PRA	VM Linux, MinIO chiffré (SSE), sauvegarde et restauration Ansible

Déploiement automatisé. Un simple envoi de code sur la branche principale, ou un déclenchement manuel, suffit à provisionner l'infrastructure, construire et analyser les images, puis déployer l'application — sans aucune manipulation à la main. Le pipeline enchaîne la connexion à Azure, l'application Terraform, la construction et la publication des images, un scan de vulnérabilités bloquant, l'installation de l'Ingress et de la supervision, le déploiement de l'application puis la configuration du stockage de sauvegarde. Il est conçu pour se rétablir seul en cas d'interruption (gestion des verrous d'état et des déploiements restés en échec).

Orchestration et résilience. L'application est packagée sous forme de chart Helm et exécutée sur Kubernetes, qui assure le redémarrage automatique des conteneurs défailants et l'ajustement du nombre de répliques en fonction de la charge. Les conteneurs s'exécutent sans privilèges, le réseau interne est cloisonné et l'accès à l'API du cluster est régi par des rôles (RBAC).

Observabilité et sécurité. L'API expose ses propres métriques, collectées par Prometheus et visualisées dans Grafana (débit, latence, taux d'erreurs, disponibilité) ; les journaux sont centralisés via Loki et les alertes routées vers une messagerie d'équipe. À chaque livraison, le code et les images sont analysés par Trivy, CodeQL et Gitleaks, et aucune image présentant une vulnérabilité critique n'est déployée.

Reprise d'activité. Les données de la base sont sauvegardées de façon chiffrée vers le stockage MinIO hébergé sur le site on-premise, et peuvent être restaurées automatiquement par Ansible. Ce mécanisme a été **validé en conditions réelles** : après avoir créé une donnée témoin, l'avoir sauvegardée puis volontairement supprimée, la restauration a permis de la retrouver à l'identique — preuve concrète de l'efficacité du plan de reprise. L'ensemble de la solution a été déployé, testé de bout en bout et jugé conforme aux attentes du cahier des charges.